

Proyecto #2 2026: Paint the Fence – Minimum Cost Interval Cover

Grupo de trabajo

Mayo 2026

1. Descripción del problema

Se modela una valla discreta con posiciones enteras desde 0 hasta L . Cada pintor i cubre un intervalo cerrado $[l_i, r_i]$ y tiene un costo positivo c_i . El objetivo es seleccionar un subconjunto de pintores cuya union cubra toda la valla y cuyo costo total sea mínimo.

Este problema es una variante clásica de cobertura de intervalos ponderada. Para los experimentos de este proyecto se asume que las coordenadas son enteras, de modo que la programación dinámica puede recorrer la valla posición por posición.

2. Análisis del problema

2.1. Subestructura óptima

Sea $dp[x]$ el costo mínimo para cubrir el prefijo $[0, x]$. Una solución óptima para $[0, x]$ termina con algún intervalo i que cubre la posición x . Si ese intervalo inicia en l_i , entonces todo el prefijo anterior $[0, l_i - 1]$ debe estar cubierto de manera óptima, porque cualquier solución mejor para ese prefijo produciría una solución mejor para $[0, x]$.

Por tanto, el problema presenta subestructura óptima.

2.2. Relación de recurrencia

La recurrencia exacta es:

$$dp[x] = \min_{i: l_i \leq x \leq r_i} (dp[l_i - 1] + c_i),$$

donde $dp[-1] = 0$. Si no existe intervalo que cubra x , entonces $dp[x] = \infty$.

3. Algoritmos de solucion

3.1. Programacion dinamica

Algorithm 1 DP exacta para cobertura de la valla

Require: Valla de longitud L , intervalos (l_i, r_i, c_i)

Ensure: Costo minimo y conjunto de intervalos elegidos

```
1: Inicializar  $dp[0..L] \leftarrow \infty$ 
2: for  $x \leftarrow 0$  to  $L$  do
3:    $mejor \leftarrow \infty$ 
4:   for cada intervalo  $i$  do
5:     if  $l_i \leq x \leq r_i$  then
6:        $prev \leftarrow 0$  si  $l_i = 0$ , en otro caso  $dp[l_i - 1]$ 
7:        $mejor \leftarrow \min(mejor, prev + c_i)$ 
8:     end if
9:   end for
10:   $dp[x] \leftarrow mejor$ 
11: end for
12: Reconstruir la solucion siguiendo los punteros guardados
```

Su tiempo de ejecucion es $O(nL)$, donde n es la cantidad de pintores y L la longitud de la valla. Como depende del valor numerico de L , es una solucion pseudo-polynomial.

3.2. Greedy

Algorithm 2 Heuristica greedy por eficiencia cobertura/costo

Require: Valla de longitud L , intervalos (l_i, r_i, c_i)

Ensure: Cobertura heuristica

```
1: Ordenar intervalos por  $l_i$ 
2:  $x \leftarrow 0$ 
3: while  $x \leq L$  do
4:   Insertar en una cola de prioridad los intervalos con  $l_i \leq x$ 
5:   Eliminar intervalos vencidos con  $r_i < x$ 
6:   Seleccionar el intervalo con mayor  $\frac{r_i - l_i + 1}{c_i}$ , rompiendo empates por mayor  $r_i$ 
7:   Agregarlo a la solucion y actualizar  $x \leftarrow r_i + 1$ 
8: end while
```

La complejidad es $O(n \log n)$: cada intervalo entra y sale de la cola de prioridad una sola vez.

4. Discusion sobre greedy choice property

Esta propiedad no se cumple en general para el problema ponderado. El criterio local de mayor eficiencia cobertura/costo puede escoger un intervalo barato y aparentemente conveniente que obliga a pagar mas en los pasos siguientes, mientras que una opcion ligeramente menos atractiva a corto plazo puede conducir a una solucion globalmente mas barata.

El archivo de resultados genera automaticamente un contraejemplo concreto y documenta que el costo de la heuristica es mayor que el de la solucion exacta.

5. Analisis empirico

Se ejecutaron ambas implementaciones sobre familias reproducibles de instancias aleatorias factibles. La semilla y los tamanos se fijaron para obtener mediciones honestas y repetibles. Cada tamano se ejecuto varias veces y se reporto la mediana de tiempos.

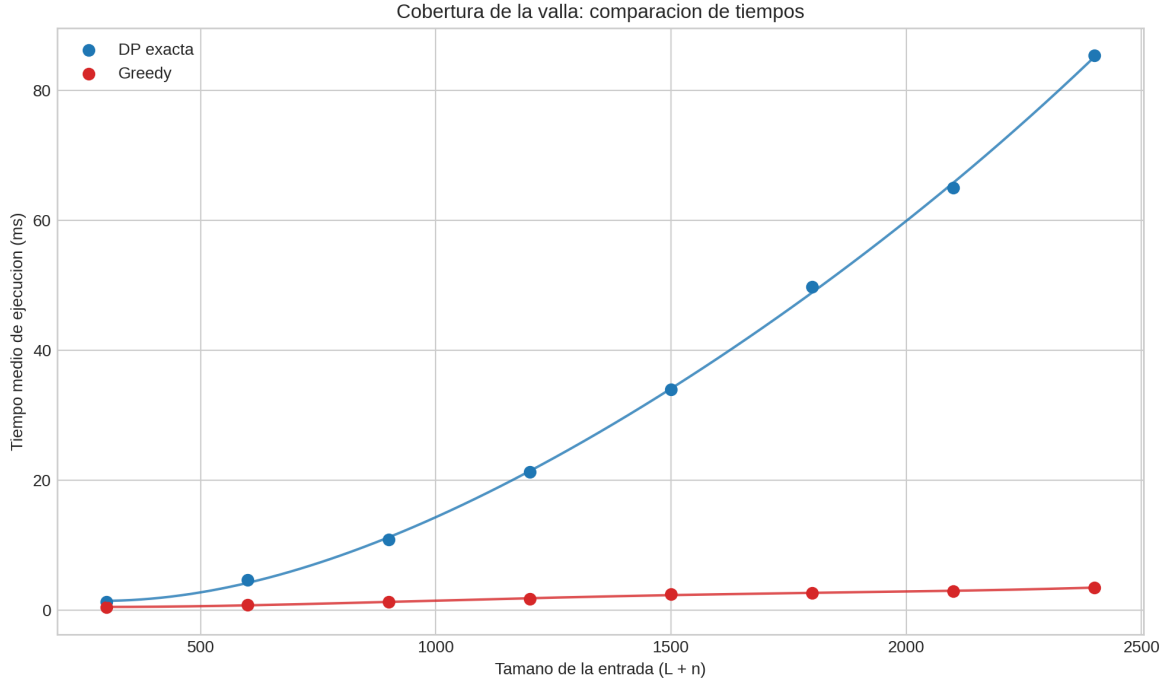


Figura 1: Tiempos de ejecucion de la DP exacta y de la heuristica greedy en funcion del tamano de entrada.

5.1. Datos usados

L	n	$L + n$	DP med. (ms)	Greedy med. (ms)	Razón media
100	200	300	1.2628	0.4609	2.8817
200	400	600	4.6802	0.7748	1.9170
300	600	900	10.8452	1.2357	2.0480
400	800	1200	21.2332	1.7236	2.6892
500	1000	1500	34.0021	2.4159	1.5252
600	1200	1800	49.8079	2.6190	2.1089
700	1400	2100	65.0147	2.9361	1.5458
800	1600	2400	85.4167	3.4707	3.3923

Counterexample usado en la discusion de greedy choice property. La instancia encontrada por busqueda aleatoria contiene la siguiente lista de intervalos:

Nombre	Izquierda	Derecha	Costo
b0	0	1	2
b1	1	2	1
b2	1	3	9

La solución óptima exacta cuesta 11 y la heurística greedy cuesta 12.

5.2. Regresión polinomial

Para cada algoritmo se ajustó una regresión polinomial y se seleccionó el mejor ajuste entre grados bajos por RMSE.

- DP exacta: grado 4, $R^2 = 0,9997$, RMSE = 0,4944, polinomio $1,85657e - 12x^4 - 1,22366e - 08x^3 + 4,1559e - 05x^2 - 0,0212443x + 4,36801$.
- Greedy: grado 4, $R^2 = 0,9963$, RMSE = 0,0609, polinomio $-3,27844e - 09x^3 + 6,18568e - 06x^2 - 0,00292239x + 0,88181$.

6. Discusión final

La programación dinámica ofrece la solución óptima, pero su costo crece con la longitud de la valla y el número de pintores. La heurística greedy es más rápida y, en promedio, produce soluciones cercanas a la óptima, pero no garantiza optimalidad. Por ello, greedy es útil cuando se prioriza velocidad y se acepta una pérdida controlada de calidad; si se requiere la mejor solución posible, la DP sigue siendo necesaria.

7. Video individual

El video debe cubrir: problema elegido, subproblemas, criterio greedy, diferencias entre greedy y DP, y la calidad observada de la heurística.